

pretrain/final-dataset — what we are building and why

For the whole team. 2026-08-07. Status: plan awaiting owner approval. Nothing ingested. Branch `final-dataset` @ `origin/edullm/perf-threaded-verify-and-gatea` (0.9.1, the deployed code).

1. The one-paragraph version

We are building a **~1.3-trillion-token pretraining corpus** for a **40B-total / 4B-active Mixture-of-Experts** model, an education-focused LLM steered on MMLU, GSM8K, ARC and HellaSwag. The corpus is **web-led, not edu-led** — that reverses our earlier plan, because DCLM-baseline measurably beats FineWeb-Edu by **+13.5 MMLU at our exact token budget**. It is delivered in **two phases**: a 1,170B main phase and a 130B cooldown that upweights math and code. Every token is stored as headerless raw `uint32` little-endian under the **dolma2 tokenizer**, which we are keeping — that choice also makes AI2's 5.93T pre-tokenized corpus a byte-copy away rather than a re-tokenization.

Why this document exists: a previous version of this plan rested on numbers that turned out to be wrong. A red-team wave refuted **five of twelve** load-bearing claims. §8 lists them. Read that section if you are going to rely on any figure here.

2. The mix — main phase, 1,170B (90% of budget)

source	share	tokens	pool	epochs	grade
DCLM-baseline	32.0%	374.4B	744.6B	0.50	MEASURED
common-pile/stackv2 (filtered)	15.0%	175.5B	707.0B	0.25	DERIVED
FineWeb-Edu	14.0%	163.8B	1,583.1B	0.10	MEASURED
FinePhrase (1 partition)	9.0%	105.3B	123.3B	0.85	MEASURED
FinePDFs-Edu	8.0%	93.6B	70.0B	1.34	CORRECTED
Nemotron-CC-Math-3+	8.0%	93.6B	133.0B	0.70	CARD ⚠
academic (peS2o+PubMed+arXiv)	4.5%	52.6B	46.6B	1.13	MEASURED
StackExchange	3.0%	35.1B	25.9B	1.36	MEASURED
Nemotron Math-Textbooks	2.5%	29.2B	27.5B	1.06	MEASURED
Cosmopedia	2.0%	23.4B	21.7B	1.08	MEASURED
reference (Wikipedia + pre-1929 books)	2.0%	23.4B	26.2B	0.89	MEASURED

Rolled up: web 54% · code 15% · synthetic 13.5% · math 8% · academic 4.5% · QA/forum 3% · reference 2%. Unique tokens consumed: **1,128B**. Max epoch: **1.36**.

3. The mix — cooldown, 130B (10% of budget)

source	share	tokens
Nemotron-CC-Math-3+	22%	28.6B
common-pile/stackv2	22%	28.6B
AI2 dolma3 100B midtraining mix	20%	26.0B
Cosmopedia	12%	15.6B
Nemotron Math-Textbooks	10%	13.0B
academic	6%	7.8B
StackExchange	5%	6.5B
reference	3%	3.9B

Code rises to 22% here on direct measurement: Aryabumi et al. (arXiv 2408.10914), **64 pretraining runs at 470M/2.8B — our scale** — found reasoning peaks at 25% code while **world knowledge is already –3.4% at 25% and –31% at 75%**. So code belongs *late and concentrated*, not spread through the main phase. AI2's 100B midtraining mix is the one published artifact purpose-built for this stage (14% QA, ~40B synthetic, 19% math, ~8M-token shards that match our granularity).

4. Why this budget: 1.3T

"1× Chinchilla" is a category error for this model. Chinchilla optimizes two free variables; our $N_{\text{total}} = 40\text{B}$ and $N_{\text{active}} = 4\text{B}$ are already fixed by memory and serving targets. What remains is monotone in D with no interior optimum. (Note also that $\sqrt{(\text{active} \times \text{total})} = 12.65\text{B}$ is *algebraically* the dense Chinchilla-optimal size at $C = 6 \cdot 4\text{B} \cdot 800\text{B}$ — so the "sqrt reading" and the "total reading" are the same claim.)

The empirical anchor is decisive: **Yuan 2.0-M32 is 40B-total / 3.7B-active — essentially our config — and shipped on 2T tokens.** The lowest from-scratch ratio in the entire small-active MoE census is 714 tokens/active-param. **We are proposing 325.** Nobody has ever shipped a 1×C small-active MoE.

At 1.3T we sit at **1.6× the "total-params" reading and below every shipped precedent** — which makes 1.3T the conservative end of defensible, not the aggressive end.

⚠ One decision to settle before this is final. Choosing 40B/4B is an inference-cost commitment — it is the only reason to pay for routing and expert parallelism. If nobody will serve this model, the better project is **a dense 12.65B on 253B tokens for ~\$15k**, skipping the MoE engineering risk entirely. That fork belongs to the team, not to this document.

5. Why web-led, not edu-led — the reversal

Our earlier plan made FineWeb-Edu the pillar at 25.5%. Three measurements say that was wrong:

- **DCLM's own Table 8, at 0.28T tokens — our budget** — gives DCLM-baseline +13.5 MMLU / +7.0 Core over FineWeb-Edu, and the gap *widens* with tokens.
- NVIDIA replicates the direction at 8B/1T: DCLM +10.5 MMLU, Nemotron-CC-HQ +16.1 MMLU.
- **The FineWeb authors' own recommended ~100B mix is 50% FinePDFs-Edu / 30% DCLM / 20% FineWeb-Edu** — their own edu flagship gets the *smallest* share.

And a structural defect we fixed. FinePhrase *is* rephrased FineWeb-Edu. Our earlier mix had 25.5% verbatim FineWeb-Edu plus 29% of its own rephrasing — **54.5% of the corpus from one upstream pool, entered twice under two labels**. No published corpus does this. This plan brings that lineage to **23%**.

Math is now one artifact, not four. Nemotron-CC-Math-3+ scores **44.20 MATH**; MegaMath-Web scores **31.60** from 263.9B tokens — *below* OpenWebMath's **34.20 from 14.7B**. And Nemotron-CC-Math sources its URLs *from* FineMath + OpenWebMath + MegaMath and refetches the WARC*s*, so it nearly contains all three. Taking it plus any of them double-counts.

Code was 9.5× under-supplied and 20× over-demanded at the same time. We had capped code at 74.81B believing that was the licensed ceiling; `common-pile/stackv2` (full, ungated, content-bearing) was never checked and is **~707B** after removing 37.9% CSV/JSON blobs. But per Aryabumi we only *want* ~35B in the main phase. **The sourcing question became a filtering question.**

6. Contamination — read this before trusting any eval number

We audited 17 corpora from papers and source code rather than dataset cards. **Nobody earns a TRUST verdict**. Two findings are directly actionable:

1. **Nemotron-CC-Math's decontamination gate effectively never fired.** An independent 13-gram scan found **11,868 contaminated documents remaining against MATH500 alone — 13.2× more than NVIDIA's entire removal budget** — including **verbatim GSM8K test items at Jaccard 1.0**. This does not change the pillar choice (every alternative scores worse) but it makes mitigation mandatory.
2. **The Common Pile ships GSM8K in Flan CoT format** (`fc-cot-cot_gsm8k` , 6 repeats, ~9× cooldown upweight). AI2 dropped Winogrande *the benchmark* rather than the data, and GSM8K was not in their suite so it never triggered the reflex. **Fix: drop one source, 0.51% of tokens.**

Our policy, four tiers:

tier	action	cost
1	exclude 9 items at source — carries nearly all the value	<1% of tokens
2	chunked embedding $\cos \geq 0.5 \rightarrow$ LLM judge, on the anneal + synthetic + question-bank slices only	\$500–2,000
3	13-gram as an instrument , not a filter — publish the number	\$10–50
4	do not do — including our own previously-decided <code>decon --ngram_size 5</code>	—

Two things everyone should know: **a quality classifier is a contamination amplifier** (~20×; the DCLM classifier puts *all* MMLU and GSM8K samples in its top-5 percentiles), and **13-gram matching scores F1 0.926 on verbatim text but exactly 0 on rephrased text** — so our 13.5% synthetic slice is structurally un-decontaminatable by n-grams.

7. How it gets built and trained

Tokenizer: dolma2, kept. No alternative clears zero measured downstream gain — SuperBPE is a null at MoE scale with a code regression, a bigger vocab is contradicted by a resolved scaling law, and `dolma2_sigdig` is a 29-minute abandoned experiment that AI2 never trained on. Keeping dolma2 also means **AI2's pre-tokenized shards are byte-identical to our format** (verified: no NumPy magic, valid ids from byte 0) — so their corpus is a copy, not a re-tokenization.

Shards: 50,003,968 tokens → ~26,000 objects. Both of my earlier objections to this were wrong: the 8192 alignment is *not* mandatory (the check is skipped unless a group declares `seq_len`, and ours does not), and mixture error is bounded by `1/shards_per_component` — **0.33% worst case, not 4.8%**.

Per-domain epochs, not uniform. phi-4 publishes an epoch column spanning **1.2× to 13.8×**, and **uniform lost head-to-head by −2.2 average, losing 6/8 benchmarks**. The mechanism is *pool size, not domain identity*. Published caps converge on 4–8×; ours peak at 1.36. Two requirements follow: `build_mixture` needs a per-source `max_epochs` enforced **inside** the optimizer (Olmix measured in-optimizer vs post-hoc as BPB 0.7647 vs 0.7855), and **weight decay must scale with $\sqrt{(\text{repeats})}$** — a one-line change measured twice, worth more than the entire quality-filtering lever.

The one genuinely MoE-specific requirement: micro-batches must not be domain-pure. Load-balancing loss computed over a domain-pure micro-batch forces uniform routing and suppresses expert specialization — worth **0.13–0.18 PPL and +5–6 GSM8k**, more than a 50% increase in activated FLOPs buys. Our shards are per-source, so **every micro-batch is domain-pure by construction**. It must be fixed **before** training: switching at 10% of the run recovers only ~55%. Everything else about the corpus we choose exactly as for a dense model — Qwen3 published that a MoE reaches dense performance *on the same data*.

Evaluation: gate on MMLU and GSM8K directly, at fixed intervals. The most reproducible harm in this literature is **knowledge collapse**, measured four independent ways (phi-4's 13.8×-synthetic arm TriviaQA – **14.8**; MoE data reuse MMLU **32.92 → 24.57**; Qwen2's 12T relaxed-quality run showing *no improvement* over 7T). **Repetition costs knowledge and spares reasoning** — exactly our weighted axis — and **no loss- or BPB-based gate can see it**. Treat a knowledge regression as a stop condition.

The one unbackfillable decision: store a per-document quality percentile as a label, at ingest. Every 2026 corpus reweights on per-document quality and topic labels, and every reweighting result worth having is unreachable without them. `entry.labels` sits inside `manifest_sha256`, so this cannot be added later without re-copying the corpus. A wrong budget is a config field; a wrong schema is a v2.

8. What we got wrong, so you can calibrate on the rest

Five of twelve load-bearing claims in the previous plan were false or void. All are corrected above.

claim	what was wrong
"8192 shard alignment is mandatory"	The check is conditional on a <code>seq_len</code> the corpus never declares
"50M shards cost 2.4%→4.8% mixture error"	Off by $\sim 15\times$. Real bound 0.33%. This was used to justify a decision
"code caps at 74.81B"	<code>common-pile/stackv2</code> was never checked. It is $\sim 707\text{B}$
"FineWeb-Edu is the pillar"	DCLM beats it by +13.5 MMLU at our budget
"77.5% Common Crawl is a problem"	Normal. Llama 1 was 82%, dolma3 76%
"min_ngram 13→5 costs -11.8 MMLU"	That is a deduplication result, not decontamination

Still unverified, and flagged rather than used: Gate A's cost at $\sim 26,000$ objects is my own extrapolation, never tested. Nemotron-CC-Math's 133B is card-grade and needs a footer count. FinePDFs-Edu was wrong by $2.3\times$ once and its $\sim 70\text{B}$ replacement is byte-derived, not directly measured. MegaMath-Web's post-filter pool size is unmeasured. Dolma 3's adult-content prevalence is **unmeasured and blocking** for any dolma3 ingest.

9. What happens next, in order

1. **Wire `keeps_id` into the build path.** It exists, is tested on 287,000 ids, and is called from nothing that writes data — so today's 59.6B of declared synthetic rests on $\sim 17\text{B}$ of real documents.
2. **Fix the missing EOS in `gigatoken-superbpe`** — `added_tokens: []` makes Gate A's EOS check skip silently in two already-published corpora.
3. **Decide the label schema** (§7, unbackfillable) and the per-source `max_epochs` API.
4. **Measure the three blocking unknowns:** Dolma 3 adult content, Nemotron-CC-Math contamination post-fix, and a real dolma2 count for Nemotron-CC-Math.
5. **Then ingest**, source by source, each as a separate approved job.

Every AWS job goes through the platform form and needs a human release. Nothing auto-publishes.